

MGN342-CA2

NAME – ANURAG PATHAK

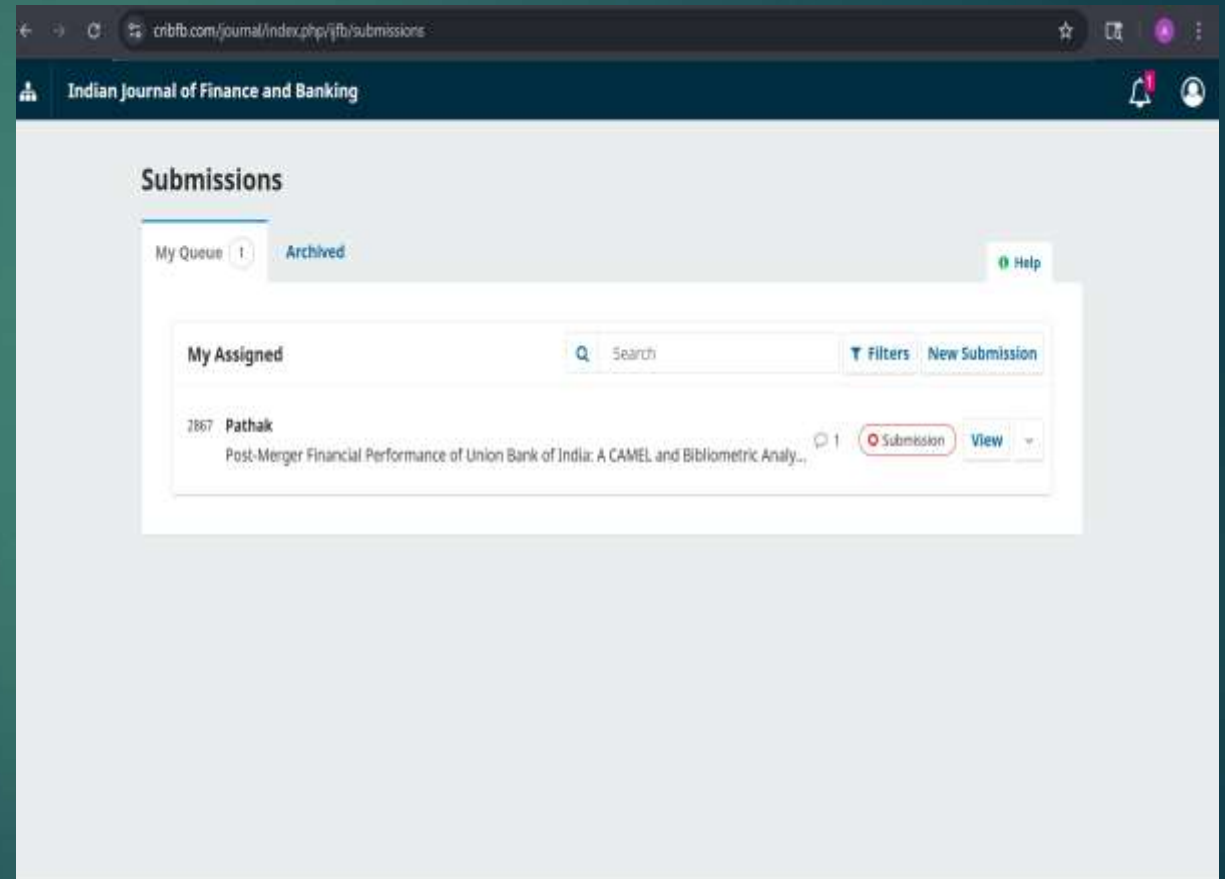
SECTION – Q2565B43

REGISTRATION NO – 12517030



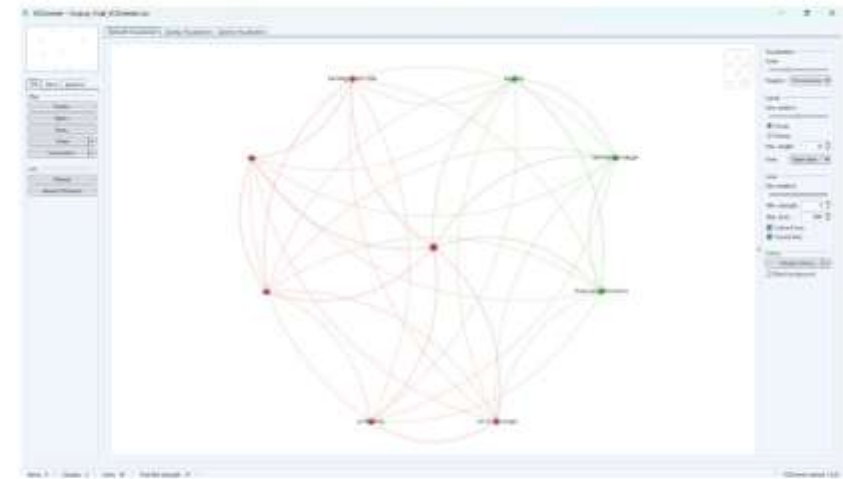
SUBMISSION AND ACCEPTANCE

- ▶ SO I HAVE POST MY DOCUMENT ON INDIAN JOURNAL OF FINANCE AND BANKING (IJFB)
- ▶ LINK - <https://www.cribfb.com/journal/index.php/ijfb/submissions>



BIBLIOMETRIC IMAGE OF UBI

- ▶ The analysis revealed strong thematic interconnections within banking research, highlighting the integrated nature of performance evaluation, regulatory frameworks, and risk management in the banking sector

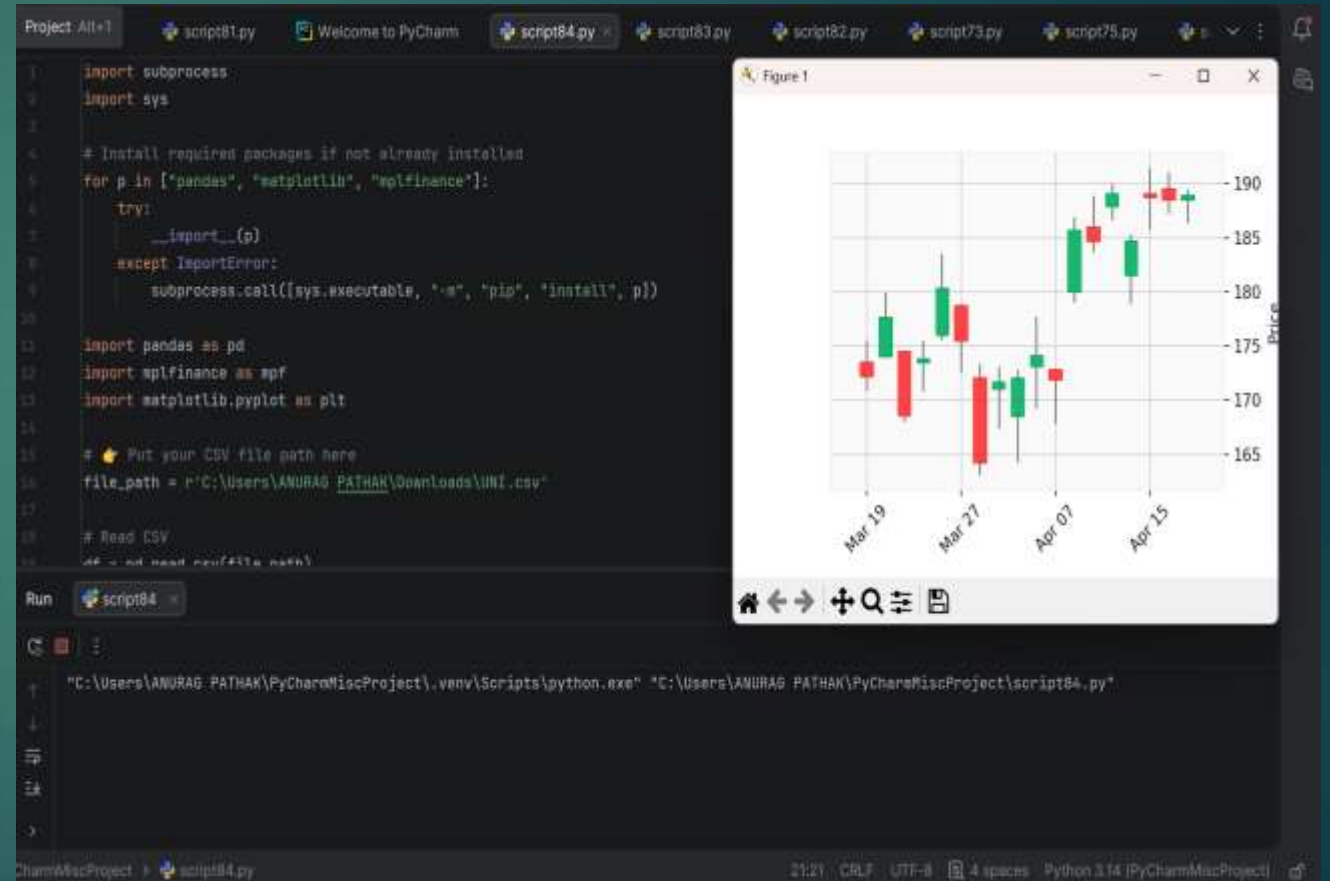


● RED CLUSTER – Profitability & Core Performance Theme

Keywords inside red cluster (from your image):

CANDKE STICK CODE

- ▶ Candlestick charts are used to analyze price movements in financial markets. Each candle represents four key values: Open, High, Low, and Close. The body of the candle shows the price range between open and close. Green candles indicate a price increase, while red candles indicate a decrease. This chart helps in identifying trends, reversals, and market sentiment.
- ▶ **LINKED IN LINK -**
https://www.linkedin.com/posts/anuragpathak0724_python-dataanalysis-finance-activity-7451575208521433089-2ryt?utm_source=share&utm_medium=member_android&rcm=ACoAAF2KrXMB20lxSYr5yKzTfoW5rRq5mlufRb0



1. How do we remove rows that failed the attention check?

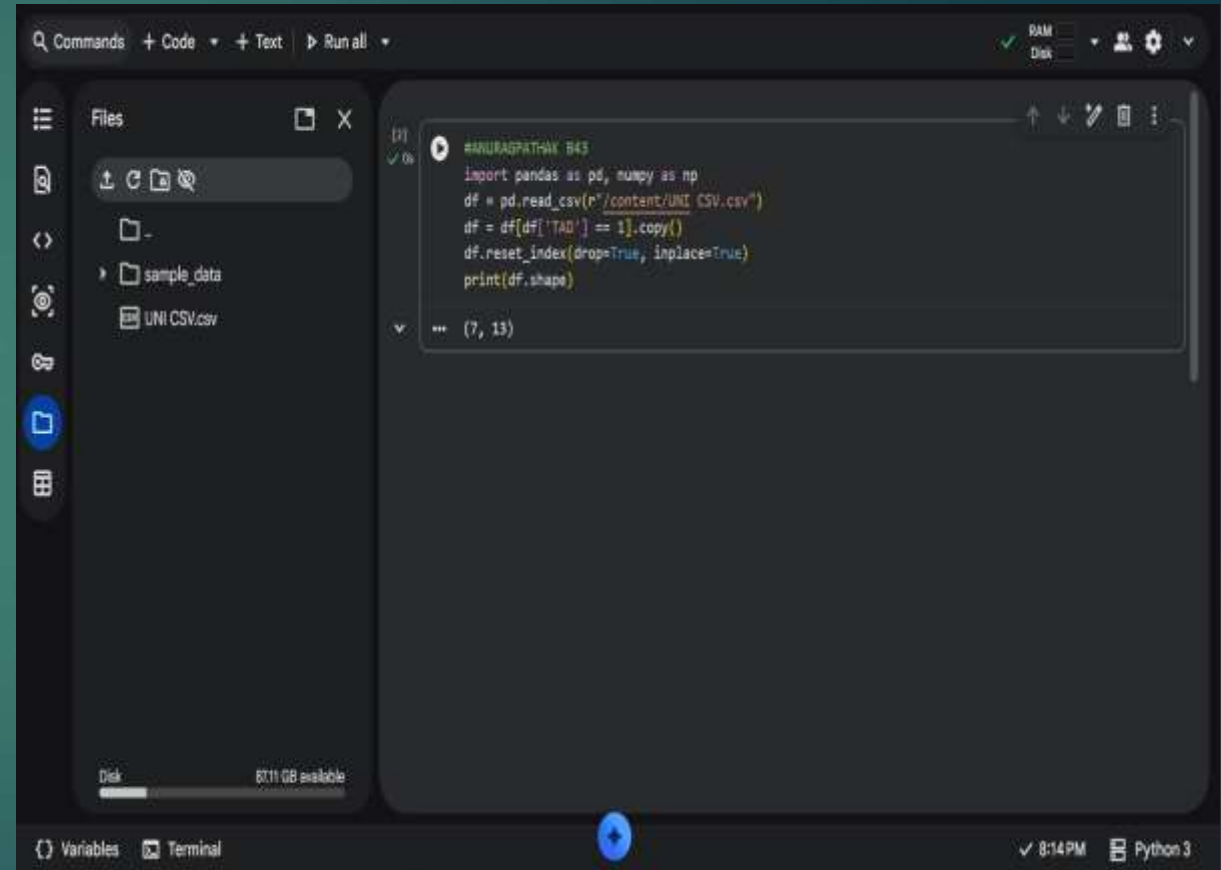
```
▶ import pandas as pd, numpy as np

▶ df =
  pd.read_csv(r"C:\D4P\Tatasteel
    .csv")

df = df[df['TAD'] == 1].copy()

df.reset_index(drop=True,
inplace=True)

print(df.shape)
```



```
Commands + Code + Text ▶ Run all ✓ RAM Disk
```

```
Files
```

-
- sample_data
- UNI CSV.csv

```
[1] ✓ 0s
```

```
#ANURAGPATHAK B43
import pandas as pd, numpy as np
df = pd.read_csv(r"/content/UNI CSV.csv")
df = df[df['TAD'] == 1].copy()
df.reset_index(drop=True, inplace=True)
print(df.shape)
```

```
... (7, 13)
```

```
{} Variables Terminal
```

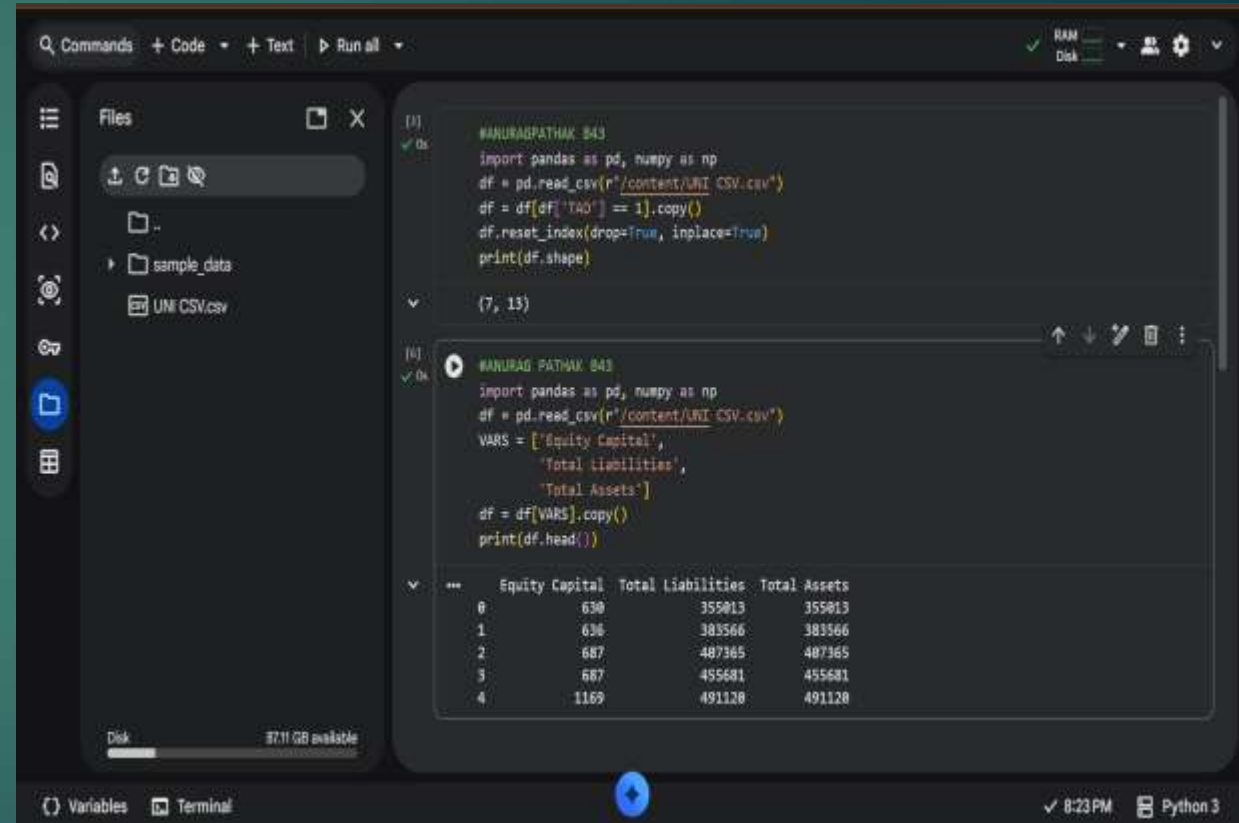
8:14 PM Python 3

2. How do we select only the variables we need?

- ▶ `import pandas as pd, numpy as np`
- ▶ `df = pd.read_csv(r"C:\D4P\Tatasteel.csv")`

```
VARs = ['EquityCapital',  
        'TotalLiabilities',  
        'TotalAssets']
```

```
df = df[VARs].copy()  
print(df.head())
```



The screenshot shows a Jupyter Notebook interface with two cells. The first cell contains the following code:

```
#ANURAG PATHAK 843  
import pandas as pd, numpy as np  
df = pd.read_csv(r"/content/UNI CSV.csv")  
df = df[df['TA0'] == 1].copy()  
df.reset_index(drop=True, inplace=True)  
print(df.shape)
```

The output of the first cell is `(7, 13)`. The second cell contains the following code:

```
#ANURAG PATHAK 843  
import pandas as pd, numpy as np  
df = pd.read_csv(r"/content/UNI CSV.csv")  
VARs = ['Equity Capital',  
        'Total Liabilities',  
        'Total Assets']  
df = df[VARs].copy()  
print(df.head())
```

The output of the second cell is a table showing the first five rows of the filtered data:

	Equity Capital	Total Liabilities	Total Assets
0	630	355013	355013
1	636	303566	303566
2	687	407365	407365
3	687	455681	455681
4	1169	491120	491120

3. How do we delete rows where more than 50% of values are missing?

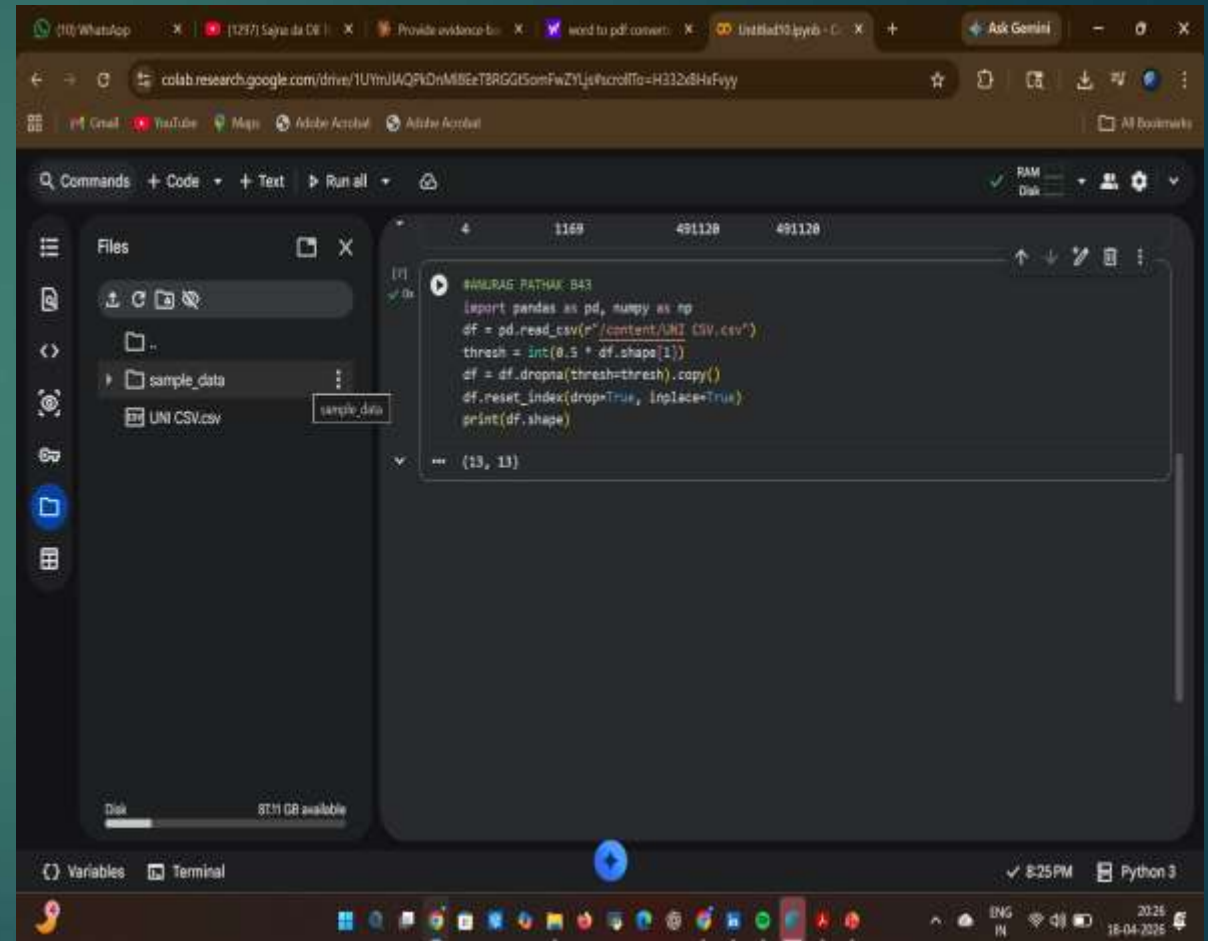
```
▶ import pandas as pd, numpy as np  
▶ df =  
  pd.read_csv(r"C:\D4P\Tatasteel  
  .csv")
```

```
thresh = int(0.5 * df.shape[1])
```

```
df =  
df.dropna(thresh=thresh).copy()
```

```
df.reset_index(drop=True,  
inplace=True)
```

```
print(df.shape)
```



The screenshot shows a Google Colab notebook interface. The browser tabs at the top include WhatsApp, a PDF converter, and the current Colab session. The address bar shows the Colab URL. The notebook's file explorer on the left shows a folder named 'sample_data' containing a file 'UNI CSV.csv'. The main code editor displays the following Python code:

```
#ANURAG PATHAK B43  
import pandas as pd, numpy as np  
df = pd.read_csv(r"/content/UNI CSV.csv")  
thresh = int(0.5 * df.shape[1])  
df = df.dropna(thresh=thresh).copy()  
df.reset_index(drop=True, inplace=True)  
print(df.shape)
```

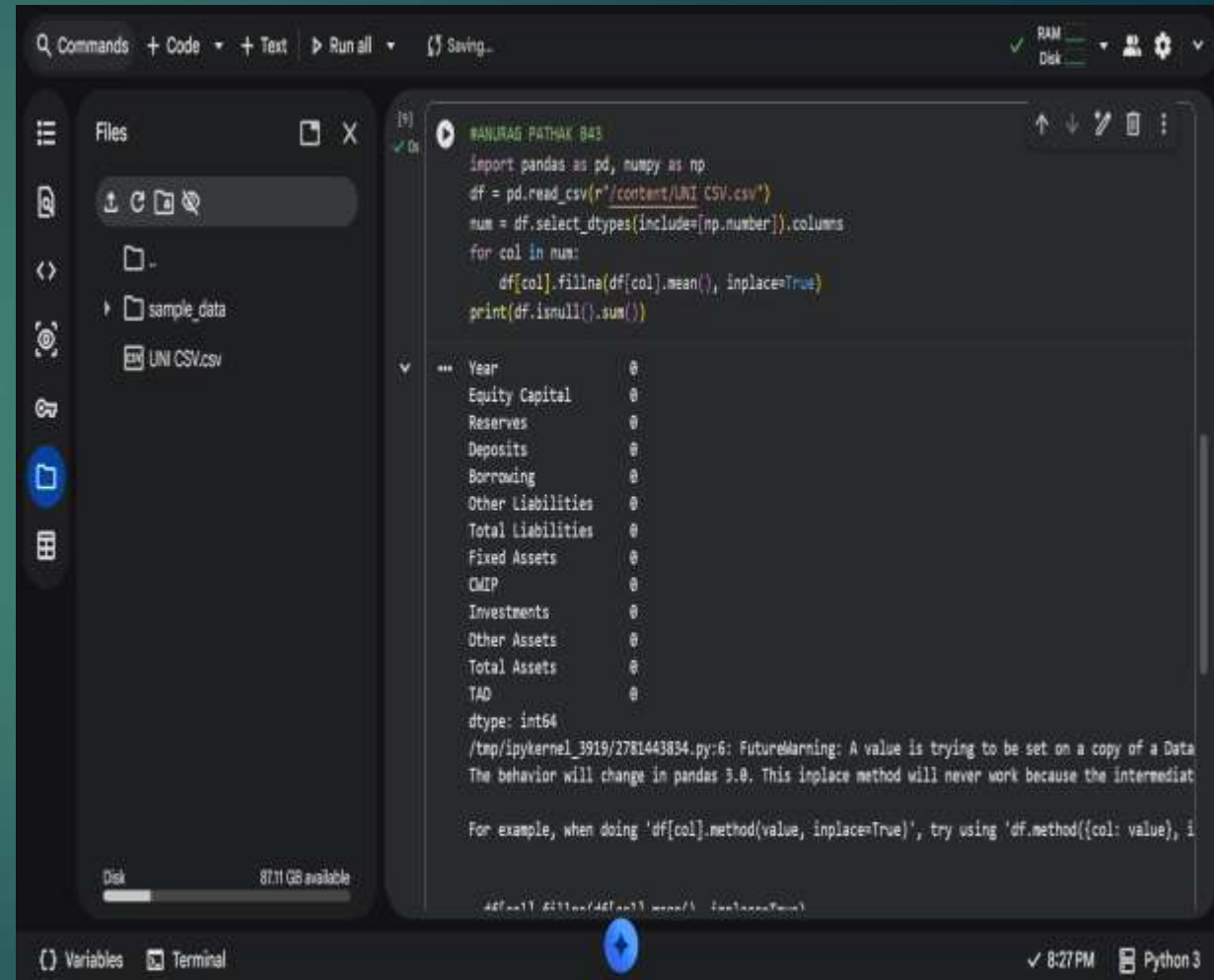
The output of the code is shown in the bottom right corner of the editor, indicating the final shape of the DataFrame: `(13, 13)`. The bottom status bar shows the system time as 8:25 PM and the Python version as Python 3.

4. How do we fill missing numeric values with the column mean?

```
▶ import pandas as pd, numpy as np
▶ df = pd.read_csv(r"C:\D4P\Tatasteel.csv")
```

```
num = df.select_dtypes(include=[np.number]).columns
for col in num:
```

```
    df[col].fillna(df[col].mean(), inplace=True)
print(df.isnull().sum())
```



The screenshot shows a Jupyter Notebook interface with a file explorer on the left, a code editor in the center, and a console output on the right. The code in the notebook is as follows:

```
#ANURAG PATHAK 843
import pandas as pd, numpy as np
df = pd.read_csv(r"/content/UNI CSV.csv")
num = df.select_dtypes(include=[np.number]).columns
for col in num:
    df[col].fillna(df[col].mean(), inplace=True)
print(df.isnull().sum())
```

The console output shows the result of the code execution:

```
Year      0
Equity Capital  0
Reserves   0
Deposits    0
Borrowing  0
Other Liabilities  0
Total Liabilities  0
Fixed Assets  0
CMIP        0
Investments  0
Other Assets  0
Total Assets  0
TAD         0
dtype: int64
```

Below the output, there is a warning message:

```
/tmp/ipykernel_3919/2781443834.py:6: FutureWarning: A value is trying to be set on a copy of a Data
The behavior will change in pandas 3.0. This inplace method will never work because the intermediat
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, i
```


The screenshot shows a Jupyter Notebook environment. On the left, a file explorer displays a folder named 'sample_data' containing a file 'UM CSV.csv'. The main workspace shows a Python script that reads a CSV file, calculates the total assets, and prints a dictionary of asset types. The output shows a dictionary with keys like 'Equity Capital', 'Reserves', 'Deposits', etc., and values like '10000', '10000', etc.

```

# [col] #index of [col] name(), inplace=True

import pandas as pd, numpy as np

df = pd.read_csv('sample_data/UM CSV.csv')
row_data = [ df['Assets'], 'Reserves', 'Total Assets' ]
for col in row_data:
    df[col] = pd.to_numeric(df[col], errors='coerce')
    print(df.dtypes)

Year
Equity Capital    int64
Reserves         int64
Deposits         int64
Borrowing        int64
Other Liabilities int64
Total Liabilities int64
Fixed Assets     int64
GDP             int64
Disinfectants    int64
Other Goods     int64
Total Assets    int64
YAB            int64
dtype: object
  
```

At the bottom, there are buttons for 'Code' and 'Text', and a status bar showing 'Variables', 'Terminal', '829PM', and 'Python 3'.

6. How do we remove duplicates and standardise column names?

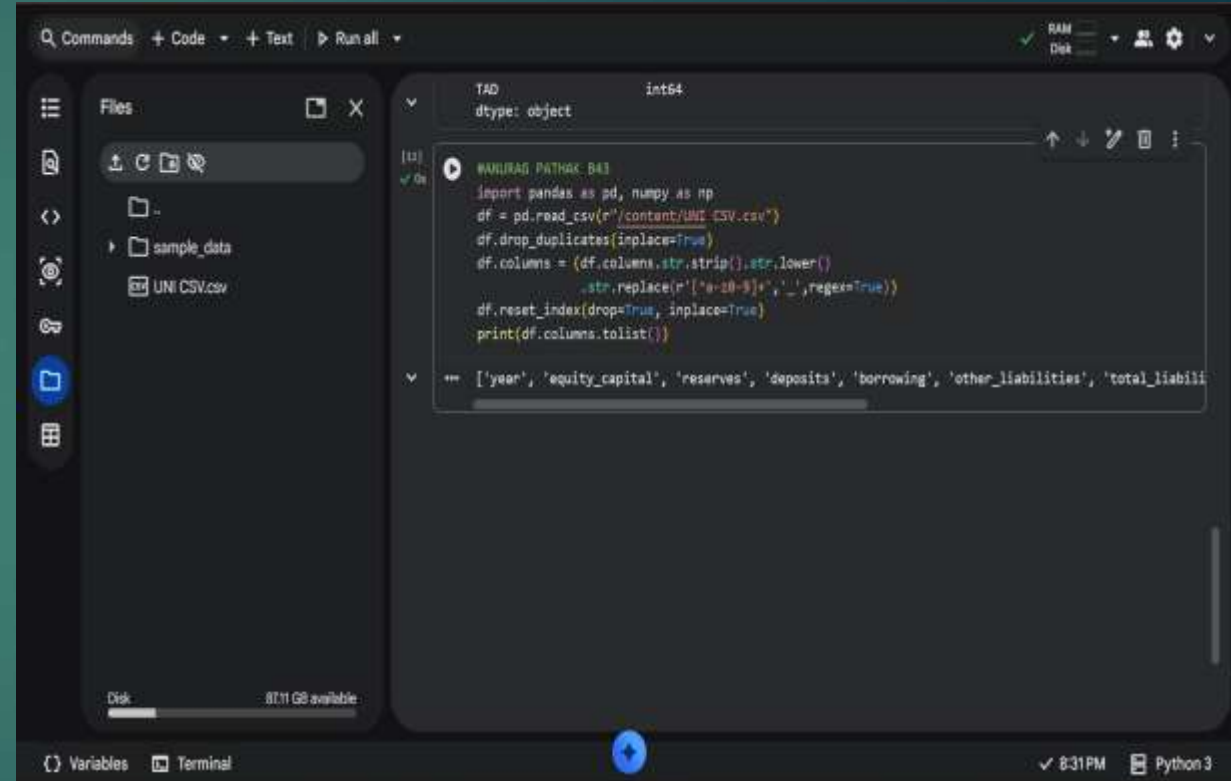
```
▶ import pandas as pd, numpy as np
▶ df =
  pd.read_csv(r"C:\D4P\Tatasteel.csv")
```

```
df.drop_duplicates(inplace=True)
```

```
df.columns =
(df.columns.str.strip().str.lower()
 .str.replace(r'^a-z0-9]+', '_', regex=True))
```

```
df.reset_index(drop=True,
inplace=True)
```

```
print(df.columns.tolist())
```



The screenshot shows a Jupyter Notebook interface with a dark theme. On the left, a file explorer shows a folder named 'sample_data' containing a file 'UNI CSV.csv'. The main area displays a code cell with the following Python code:

```
import pandas as pd, numpy as np
df = pd.read_csv(r"C:\content\UNI CSV.csv")
df.drop_duplicates(inplace=True)
df.columns = (df.columns.str.strip().str.lower()
              .str.replace(r'^a-z0-9]+', '_', regex=True))
df.reset_index(drop=True, inplace=True)
print(df.columns.tolist())
```

Below the code, the output of the `print` statement is visible, showing a list of column names: `['year', 'equity_capital', 'reserves', 'deposits', 'borrowing', 'other_liabilities', 'total_liabili']`. The interface also shows a 'Variables' panel on the left and a 'Terminal' panel at the bottom.

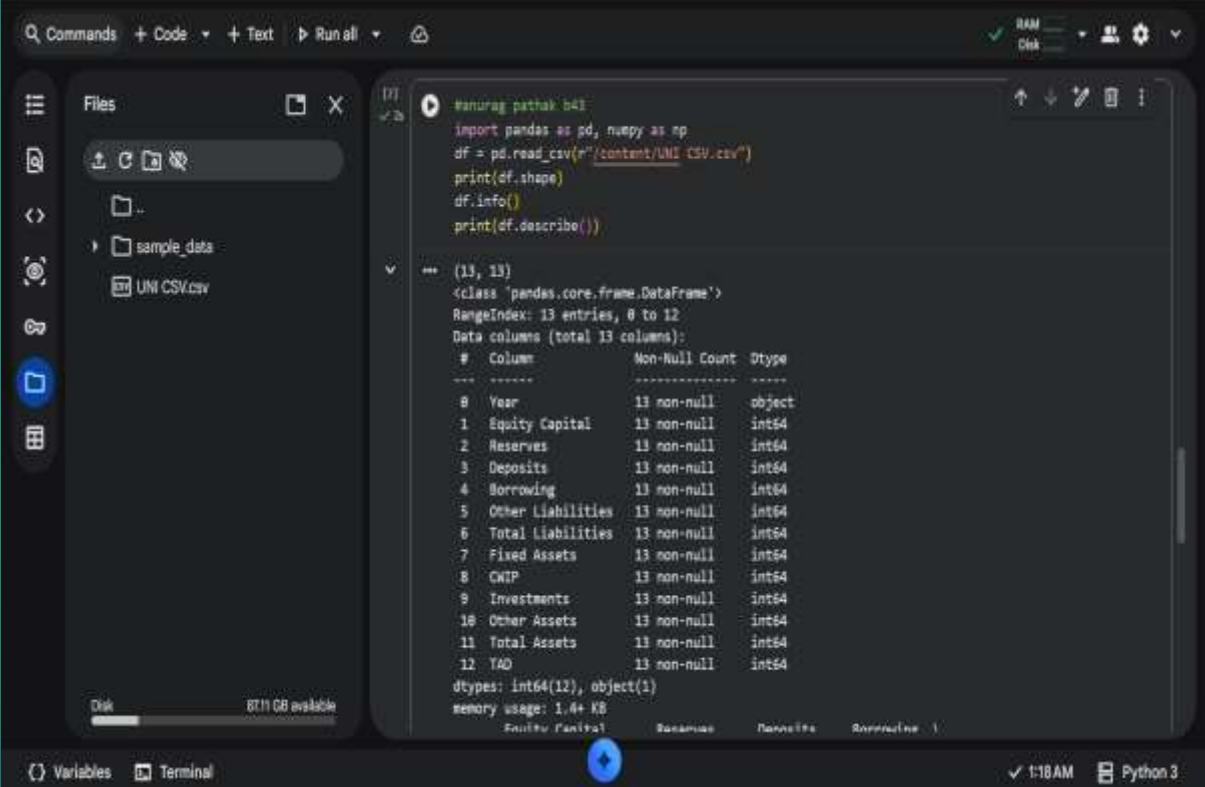
7. How do we get a summary of the dataset's structure?

- ▶ `import pandas as pd, numpy as np`
- ▶ `df = pd.read_csv(r"C:\D4P\Tatasteel.csv")`

`print(df.shape)`

`df.info()`

`print(df.describe())`



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The file explorer shows a folder named 'sample_data' containing a file 'UNI CSV.csv'. The code editor contains the following Python code:

```
import pandas as pd, numpy as np
df = pd.read_csv(r"C:\content\UNI CSV.csv")
print(df.shape)
df.info()
print(df.describe())
```

The output of the code is displayed below the code cells. It shows the shape of the DataFrame as (13, 13), the class as pandas.core.frame.DataFrame, the RangeIndex as 13 entries, 0 to 12, and the Data columns (total 13 columns) as follows:

#	Column	Non-Null Count	Dtype
0	Year	13 non-null	object
1	Equity Capital	13 non-null	int64
2	Reserves	13 non-null	int64
3	Deposits	13 non-null	int64
4	Borrowing	13 non-null	int64
5	Other Liabilities	13 non-null	int64
6	Total Liabilities	13 non-null	int64
7	Fixed Assets	13 non-null	int64
8	CWIP	13 non-null	int64
9	Investments	13 non-null	int64
10	Other Assets	13 non-null	int64
11	Total Assets	13 non-null	int64
12	YTD	13 non-null	int64

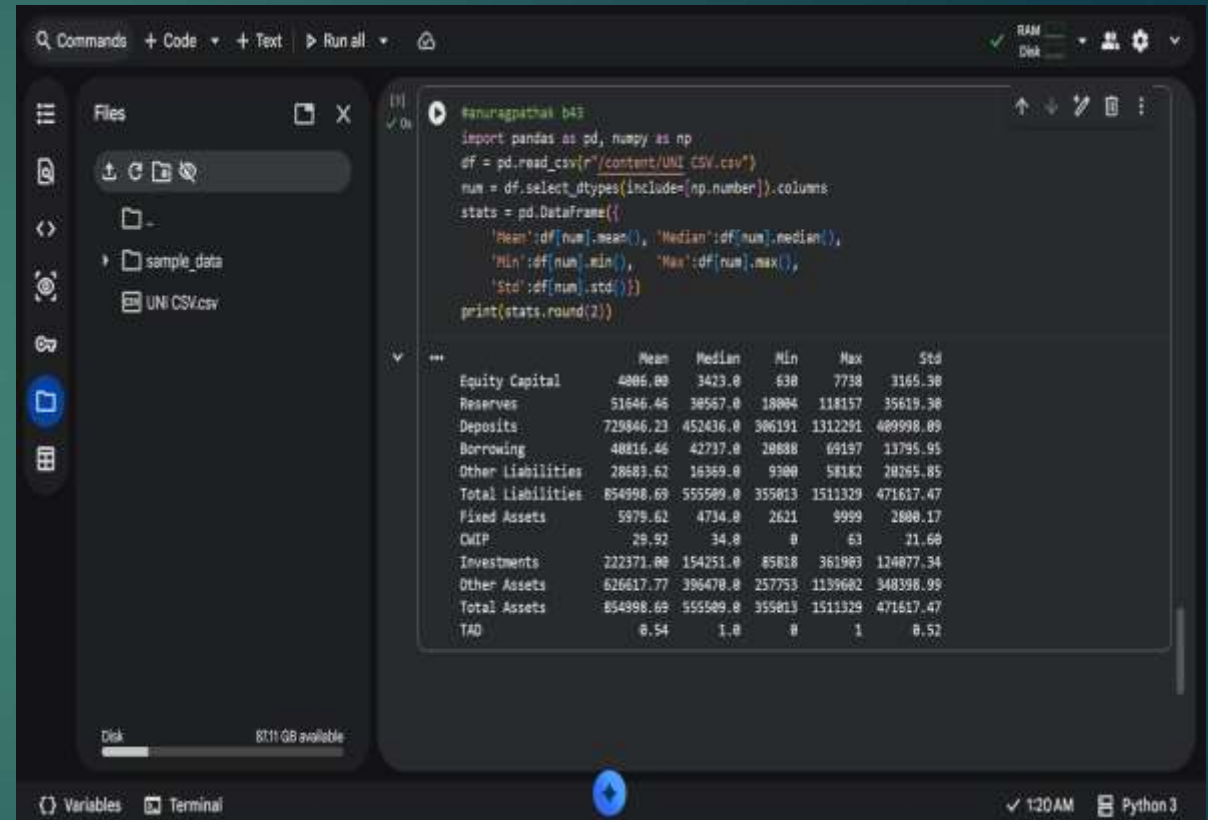
The dtypes are int64(12), object(1) and the memory usage is 1.4+ KB.

8. How do we compute mean, median, min, max and std deviation?

```
▶ import pandas as pd, numpy as np
▶ df =
  pd.read_csv(r"C:\D4P\Tatasteel.csv")
```

```
num =
df.select_dtypes(include=[np.number
]).columns
```

```
stats = pd.DataFrame({
    'Mean':df[num].mean(),
    'Median':df[num].median(),
    'Min':df[num].min(),
    'Max':df[num].max(),
    'Std':df[num].std()})
print(stats.round(2))
```



```
#anuragpathak b43
import pandas as pd, numpy as np
df = pd.read_csv(r"/content/UNI CSV.csv")
num = df.select_dtypes(include=[np.number]).columns
stats = pd.DataFrame({
    'Mean':df[num].mean(), 'Median':df[num].median(),
    'Min':df[num].min(), 'Max':df[num].max(),
    'Std':df[num].std()})
print(stats.round(2))
```

	Mean	Median	Min	Max	Std
Equity Capital	4886.88	3423.0	638	7738	3165.38
Reserves	51646.46	38567.0	18004	118157	35619.38
Deposits	729846.23	452436.0	386191	1312291	489998.89
Borrowing	48816.46	42737.0	28888	69197	13795.95
Other Liabilities	28889.62	16369.0	9900	58182	28265.85
Total Liabilities	854998.69	555589.0	355813	1511329	471617.47
Fixed Assets	5979.62	4734.0	2621	9999	2880.17
OWP	29.92	34.0	0	63	21.68
Investments	222371.88	154251.0	85818	361983	124877.34
Other Assets	626617.77	396478.0	257753	1139682	348398.99
Total Assets	854998.69	555589.0	355813	1511329	471617.47
TAD	0.54	1.0	0	1	0.52

9. How do we create new derived columns from existing ones?

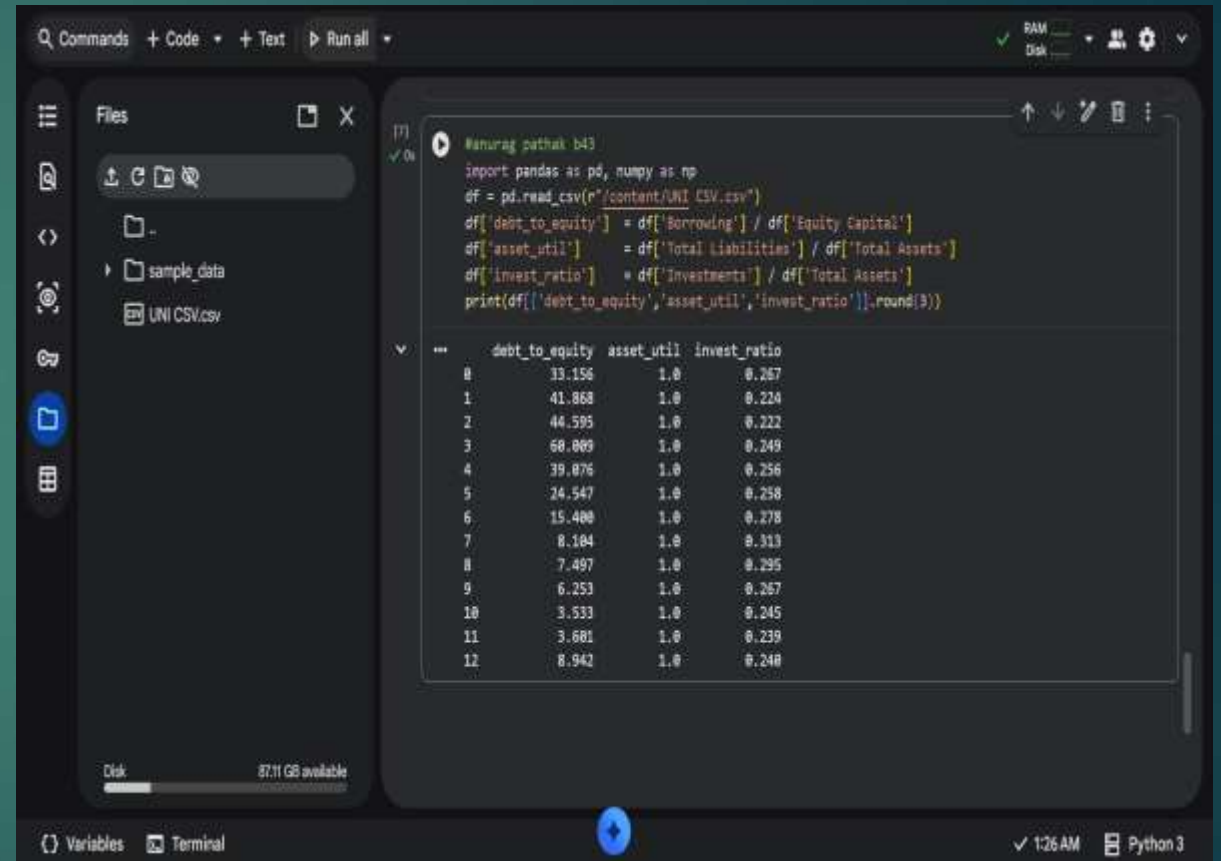
```
▶ import pandas as pd, numpy as np
▶ df =
  pd.read_csv(r"C:\D4P\Tatasteel.csv")
```

```
df['debt_to_equity'] =
df['Borrowings +'] /
df['EquityCapital']
```

```
df['asset_util'] =
df['TotalLiabilities'] /
df['TotalAssets']
```

```
df['invest_ratio'] =
df['Investments'] /
df['TotalAssets']
```

```
print(df[['debt_to_equity', 'asset_util', 'invest_ratio']].round(3))
```



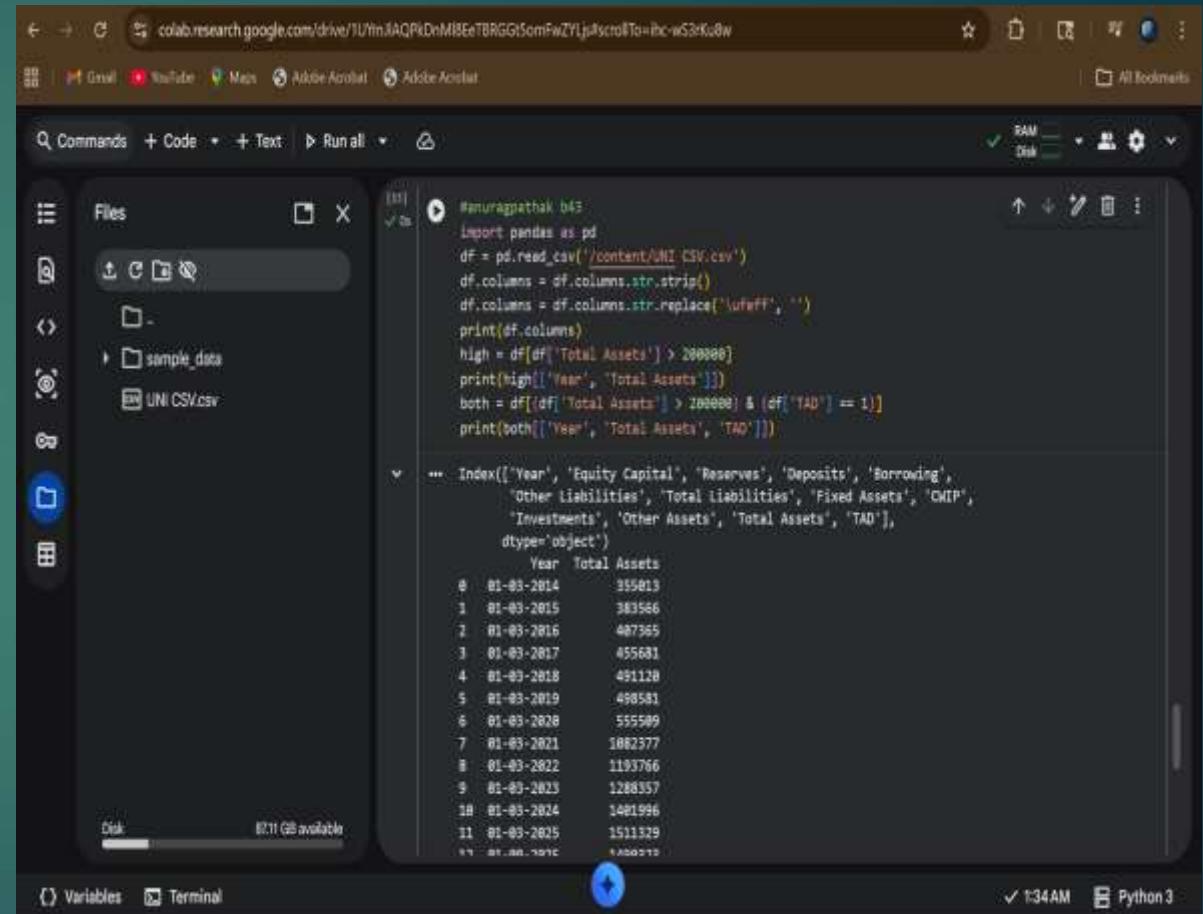
The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The code in the editor defines three new columns: 'debt_to_equity', 'asset_util', and 'invest_ratio'. Below the code, the output of the print statement is displayed as a table with 12 rows of data.

```
import pandas as pd, numpy as np
df = pd.read_csv(r"C:\content\UNI CSV.csv")
df['debt_to_equity'] = df['Borrowing'] / df['Equity Capital']
df['asset_util'] = df['Total Liabilities'] / df['Total Assets']
df['invest_ratio'] = df['Investments'] / df['Total Assets']
print(df[['debt_to_equity', 'asset_util', 'invest_ratio']].round(3))
```

	debt_to_equity	asset_util	invest_ratio
0	33.156	1.0	0.267
1	41.868	1.0	0.224
2	44.595	1.0	0.222
3	60.009	1.0	0.249
4	39.076	1.0	0.256
5	24.547	1.0	0.258
6	15.400	1.0	0.278
7	8.104	1.0	0.313
8	7.497	1.0	0.295
9	6.253	1.0	0.267
10	3.533	1.0	0.245
11	3.681	1.0	0.239
12	8.942	1.0	0.240

10. How do we filter rows based on a condition?

```
▶ import pandas as  
pd, numpy as np  
▶ df =  
pd.read_csv(r"C:\D  
4P\Tatasteel.csv")  
  
high = df[df['TotalAssets'] >  
200000]  
  
print(high[['Year', 'TotalAssets']])  
  
both = df[(df['TotalAssets'] >  
200000) & (df['TAD'] == 1)]  
  
print(both[['Year', 'TotalAssets', 'T  
AD']])
```



The screenshot shows a Google Colab notebook interface. The left sidebar displays the file explorer with a folder named 'sample_data' and a file named 'UNI CSV.csv'. The main area shows the code editor with the following Python code:

```
#anuragpathak b43  
import pandas as pd  
df = pd.read_csv('/content/UNI CSV.csv')  
df.columns = df.columns.str.strip()  
df.columns = df.columns.str.replace('\ufeff', '')  
print(df.columns)  
high = df[df['Total Assets'] > 200000]  
print(high[['Year', 'Total Assets']])  
both = df[(df['Total Assets'] > 200000) & (df['TAD'] == 1)]  
print(both[['Year', 'Total Assets', 'TAD']])
```

Below the code editor, the output of the last print statement is displayed as a table:

	Year	Total Assets
0	01-03-2014	355019
1	01-03-2015	383566
2	01-03-2016	407365
3	01-03-2017	455681
4	01-03-2018	481120
5	01-03-2019	498581
6	01-03-2020	555509
7	01-03-2021	1082377
8	01-03-2022	1193766
9	01-03-2023	1288357
10	01-03-2024	1401996
11	01-03-2025	1511329

The bottom of the interface shows the 'Variables' and 'Terminal' tabs, and the status bar indicates '1:34 AM Python 3'.

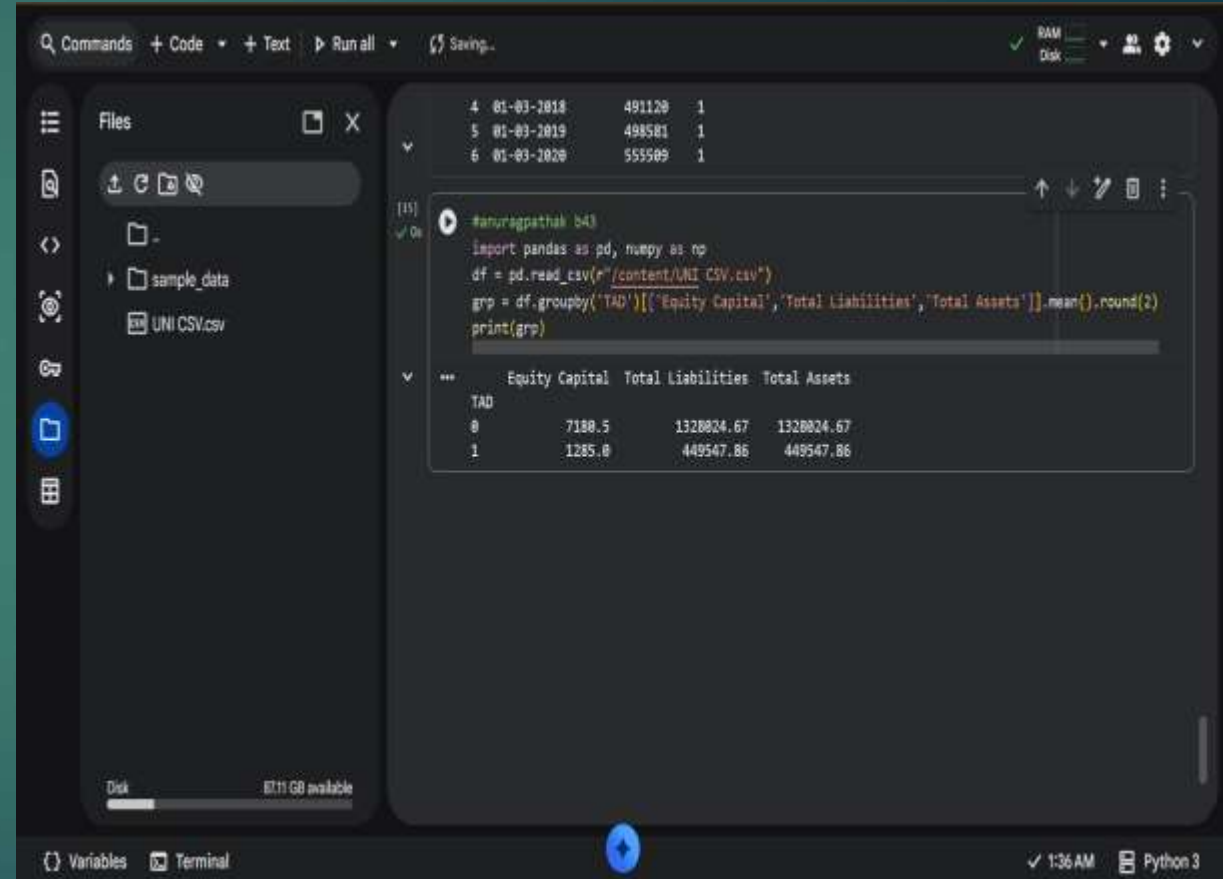
11. How do we compute group-wise statistics using groupby?

```
▶ import pandas as pd, numpy as np

▶ df =
    pd.read_csv(r"C:\D4P\Tatasteel.csv")

grp =
df.groupby('TAD')[['EquityCapital',
'TotalLiabilities','TotalAssets']].
mean().round(2)

print(grp)
```



```
Commands + Code + Text Run all Saving... RAM Disk
```

Files

-
- sample_data
- UNI CSV.csv

```
[15] #anuregpathak b41
import pandas as pd, numpy as np
df = pd.read_csv(r"/content/UNI CSV.csv")
grp = df.groupby('TAD')[['Equity Capital', 'Total Liabilities', 'Total Assets']].mean().round(2)
print(grp)
```

	Equity Capital	Total Liabilities	Total Assets
TAD			
0	7188.5	1328824.67	1328824.67
1	1285.8	449547.86	449547.86

Disk 87.11 GB available

Variables Terminal 1:36 AM Python 3